

Laboratory work: Comparison of sorting algorithms

1.0

Generated by Doxygen 1.16.1

1 Лабораторная работа: Сравнение алгоритмов сортировки	1
1.1 Автор	1
1.2 Ссылка на исходный код	1
1.3 Задание	1
1.4 Таблица времени выполнения (микросекунды)	2
1.5 Графики	2
1.6 Анализ результатов	2
1.7 Выводы (какой метод когда применять)	3
2 Class Index	5
2.1 Class List	5
3 File Index	7
3.1 File List	7
4 Class Documentation	9
4.1 Flight Class Reference	9
4.1.1 Detailed Description	10
4.1.2 Constructor & Destructor Documentation	10
4.1.2.1 Flight() [1/2]	10
4.1.2.2 Flight() [2/2]	10
4.1.3 Member Function Documentation	11
4.1.3.1 getAirline()	11
4.1.3.2 getArrivalDate()	11
4.1.3.3 getArrivalTime()	11
4.1.3.4 getFlightNumber()	11
4.1.3.5 getPassengers()	11
4.1.3.6 operator<()	12
4.1.3.7 operator>()	12
4.1.3.8 toCSV()	13
4.1.4 Friends And Related Symbol Documentation	13
4.1.4.1 operator<<	13
4.1.4.2 operator>>	13
4.1.5 Member Data Documentation	14
4.1.5.1 airline	14
4.1.5.2 arrivalDate	14
4.1.5.3 arrivalTime	14
4.1.5.4 flightNumber	14
4.1.5.5 passengers	14
4.2 FlightData Struct Reference	14
4.2.1 Detailed Description	15
4.2.2 Member Data Documentation	15
4.2.2.1 airline	15
4.2.2.2 arrivalDate	15

4.2.2.3	arrivalTime	15
4.2.2.4	flightNumber	15
4.2.2.5	passengers	15
5	File Documentation	17
5.1	generate_data.cpp File Reference	17
5.1.1	Detailed Description	18
5.1.2	Использование	18
5.1.3	Выходные данные	18
5.1.4	Function Documentation	18
5.1.4.1	gen()	18
5.1.4.2	generateFlight()	19
5.1.4.3	main()	19
5.1.4.4	randomAirline()	19
5.1.4.5	randomDate()	19
5.1.4.6	randomFlightNumber()	20
5.1.4.7	randomPassengers()	20
5.1.4.8	randomTime()	20
5.1.4.9	saveToCSV()	20
5.1.5	Variable Documentation	21
5.1.5.1	AIRLINES	21
5.1.5.2	PREFIXES	21
5.1.5.3	rd	21
5.2	main.cpp File Reference	21
5.2.1	Function Documentation	22
5.2.1.1	bubbleSort()	22
5.2.1.2	isSorted()	23
5.2.1.3	loadFromCSV()	24
5.2.1.4	main()	24
5.2.1.5	measureTime()	25
5.2.1.6	partition()	25
5.2.1.7	quickSort()	26
5.2.1.8	quickSortRecursive()	27
5.2.1.9	saveResultToFile()	27
5.2.1.10	shakerSort()	28

Глава 1

Лабораторная работа: Сравнение алгоритмов сортировки

1.1 Автор

ФИО: Починава Алина Алексеевна Группа: СКБ231 Вариант: 21

1.2 Ссылка на исходный код

GitHub репозиторий: <https://github.com/alinapochinova/sorting>

1.3 Задание

Реализовать и сравнить алгоритмы сортировки:

- Сортировка пузырьком (Bubble sort)
- Шейкер-сортировка (Shaker sort)
- Быстрая сортировка (Quick sort)
- Стандартная сортировка `std::sort`

Объект сортировки: Авиарейсы (номер, авиакомпания, дата прилета, время прилета, число пассажиров) Сравнение по: дата → время → авиакомпания → число пассажиров (по убыванию)

1.4 Таблица времени выполнения (микросекунды)

Размер	Bubble	Shaker	Quick	std::sort
100	829	357	50	61
500	11466	6798	308	1757
1000	36279	24792	681	478
5000	596974	530758	3808	5820
10000	2507160	1984311	7750	7006
20000	9507980	8041423	13315	14903
30000	21811392	18265228	24309	21105
50000	61036144	51655407	36236	35143
75000	138588085	122068333	94687	55282
100000	246126707	206496517	70776	65607

1.5 Графики

Пояснение к графику 1:

- Левый график показывает работу квадратичных алгоритмов (Bubble, Shaker) — время растёт очень быстро
- Правый график показывает работу быстрых алгоритмов (Quick, std::sort) — время растёт медленно
- Разница в скорости между левым и правым графиком достигает 1000+ раз на больших размерах

Пояснение к графику 2:

- Логарифмическая шкала позволяет увидеть все алгоритмы на одном графике
- Bubble и Shaker имеют наклон 2 (соответствует $O(n^2)$)
- Quick и std::sort имеют наклон 1 (соответствует $O(n \log n)$)

1.6 Анализ результатов

Наблюдения:

1. Bubble и Shaker демонстрируют квадратичный рост $O(n^2)$
2. Quick sort и std::sort показывают линейно-логарифмический рост $O(n \log n)$
3. std::sort на 5-15% быстрее Quick sort на всех размерах
4. На размере 100000 Bubble работает более 350 секунд, а std::sort — менее 0.1 секунды

1.7 Выводы (какой метод когда применять)

Ситуация	Рекомендуемый алгоритм	Обоснование
Маленькие массивы (< 1000)	Bubble или Shaker	Простота реализации, разница времени незначительна
Большие массивы (> 10000)	Quick sort или std::sort	$O(n \log n)$ против $O(n^2)$ — разница в тысячи раз
Требуется устойчивая сортировка	Bubble или Shaker	Quick sort нестабилен
Данные почти отсортированы	Shaker sort	Естественное поведение (быстро завершается)
Производственный код	std::sort	Оптимизирован, гарантированная производительность
Образовательные цели	Bubble sort	Простейший для понимания алгоритм

Общий вывод:

- Стандартная сортировка std::sort — лучший выбор для реальных задач
- Quick sort — хорошая альтернатива, если std::sort недоступен
- Пузырьковые алгоритмы (Bubble, Shaker) применимы только для обучения или очень маленьких массивов (< 1000 элементов)

See also

[Flight](#)
[bubbleSort](#)
[shakerSort](#)
[quickSort](#)

Глава 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Flight	Класс, представляющий информацию об авиарейсе	9
FlightData	Вспомогательная структура для генерации данных	14

Глава 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

generate_data.cpp	
Генератор тестовых данных для лабораторной работы	17
main.cpp	21

Глава 4

Class Documentation

4.1 Flight Class Reference

Класс, представляющий информацию об авиарейсе

Public Member Functions

- `Flight ()`
Конструктор по умолчанию
- `Flight (const string &fn, const string &al, const string &date, const string &time, int pass)`
Конструктор с параметрами
- `string getFlightNumber () const`
- `string getAirline () const`
- `string getArrivalDate () const`
- `string getArrivalTime () const`
- `int getPassengers () const`
- `bool operator< (const Flight &other) const`
Оператор "меньше" для сортировки
- `bool operator> (const Flight &other) const`
Оператор "больше".
- `string toCSV () const`
Преобразование в строку CSV (без заголовка).

Private Attributes

- `string flightNumber`
Номер рейса (например, SU1234).
- `string airline`
Название авиакомпании
- `string arrivalDate`
Дата прилета (YYYY-MM-DD).
- `string arrivalTime`
Время прилета (HH:MM:SS).
- `int passengers`
Число пассажиров на борту (0-300).

Friends

- ostream & [operator<<](#) (ostream &os, const [Flight](#) &f)
Оператор вывода в поток
- istream & [operator>>](#) (istream &is, [Flight](#) &f)
Оператор ввода из потока (чтение CSV).

4.1.1 Detailed Description

Класс, представляющий информацию об авиарейсе

Содержит все необходимые поля для описания рейса, а также перегруженные операторы сравнения для сортировки.

Invariant

passengers >= 0 (число пассажиров неотрицательное)
arrivalDate в формате YYYY-MM-DD
arrivalTime в формате HH:MM:SS

4.1.2 Constructor & Destructor Documentation

4.1.2.1 [Flight\(\)](#) [1/2]

[Flight::Flight](#) () [inline]

Конструктор по умолчанию

Инициализирует все поля пустыми значениями

4.1.2.2 [Flight\(\)](#) [2/2]

[Flight::Flight](#) (
 const string & fn,
 const string & al,
 const string & date,
 const string & time,
 int pass) [inline]

Конструктор с параметрами

Parameters

fn	Номер рейса
al	Название авиакомпании
date	Дата прилета (формат YYYY-MM-DD)
time	Время прилета (формат HH:MM:SS)

pass	Число пассажиров
------	------------------

4.1.3 Member Function Documentation

4.1.3.1 getAirline()

```
string Flight::getAirline () const [inline]
```

Returns

Название авиакомпании

4.1.3.2 getArrivalDate()

```
string Flight::getArrivalDate () const [inline]
```

Returns

Дата прилета

4.1.3.3 getArrivalTime()

```
string Flight::getArrivalTime () const [inline]
```

Returns

Время прилета

4.1.3.4 getFlightNumber()

```
string Flight::getFlightNumber () const [inline]
```

Returns

Номер рейса

4.1.3.5 getPassengers()

```
int Flight::getPassengers () const [inline]
```

Returns

Число пассажиров

4.1.3.6 operator<()

```
bool Flight::operator< (  
    const Flight & other) const    [inline]
```

Оператор "меньше" для сортировки

Parameters

other	Объект для сравнения
-------	----------------------

Returns

true если текущий объект должен идти раньше other

Правила сравнения (по приоритету):

1. Сравнение по дате прилета
2. При равенстве дат - по времени прилета
3. При равенстве времени - по названию авиакомпании
4. При равенстве авиакомпании - по числу пассажиров (по убыванию!)

Note

Число пассажиров сравнивается в обратном порядке (убывание), чтобы при сортировке по возрастанию рейсы с большим числом пассажиров шли первыми.

4.1.3.7 operator>()

```
bool Flight::operator> (  
    const Flight & other) const    [inline]
```

Оператор "больше".

Parameters

other	Объект для сравнения
-------	----------------------

Returns

true если текущий объект больше other

Note

Реализован через оператор <

4.1.3.8 toCSV()

```
string Flight::toCSV () const [inline]
```

Преобразование в строку CSV (без заголовка).

Returns

Строка вида "номер,компания,дата,время,пассажиры"

4.1.4 Friends And Related Symbol Documentation

4.1.4.1 operator<<

```
ostream & operator<< (  
    ostream & os,  
    const Flight & f) [friend]
```

Оператор вывода в поток

Parameters

os	Поток вывода
f	Объект Flight

Returns

Ссылка на поток для цепочечных операций

4.1.4.2 operator>>

```
istream & operator>> (  
    istream & is,  
    Flight & f) [friend]
```

Оператор ввода из потока (чтение CSV).

Parameters

is	Поток ввода
f	Объект Flight для заполнения

Returns

Ссылка на поток для цепочечных операций

Ожидает строку формата: "номер,компания,дата,время,пассажиры"

4.1.5 Member Data Documentation

4.1.5.1 airline

`string Flight::airline [private]`

Название авиакомпании

4.1.5.2 arrivalDate

`string Flight::arrivalDate [private]`

Дата прилета (YYYY-MM-DD).

4.1.5.3 arrivalTime

`string Flight::arrivalTime [private]`

Время прилета (HH:MM:SS).

4.1.5.4 flightNumber

`string Flight::flightNumber [private]`

Номер рейса (например, SU1234).

4.1.5.5 passengers

`int Flight::passengers [private]`

Число пассажиров на борту (0-300).

The documentation for this class was generated from the following file:

- [main.cpp](#)

4.2 FlightData Struct Reference

Вспомогательная структура для генерации данных

Public Attributes

- string [flightNumber](#)
Номер рейса
- string [airline](#)
Авиакомпания
- string [arrivalDate](#)
Дата прилета (YYYY-MM-DD).
- string [arrivalTime](#)
Время прилета (HH:MM:SS).
- int [passengers](#)
Число пассажиров (0-300).

4.2.1 Detailed Description

Вспомогательная структура для генерации данных

Используется только в процессе генерации, не участвует в сортировке

4.2.2 Member Data Documentation

4.2.2.1 [airline](#)

string FlightData::airline

Авиакомпания

4.2.2.2 [arrivalDate](#)

string FlightData::arrivalDate

Дата прилета (YYYY-MM-DD).

4.2.2.3 [arrivalTime](#)

string FlightData::arrivalTime

Время прилета (HH:MM:SS).

4.2.2.4 [flightNumber](#)

string FlightData::flightNumber

Номер рейса

4.2.2.5 [passengers](#)

int FlightData::passengers

Число пассажиров (0-300).

The documentation for this struct was generated from the following file:

- [generate_data.cpp](#)

Глава 5

File Documentation

5.1 generate_data.cpp File Reference

Генератор тестовых данных для лабораторной работы

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <random>
#include <iomanip>
#include <chrono>
```

Classes

- struct [FlightData](#)
Вспомогательная структура для генерации данных

Functions

- [mt19937 gen](#) ([rd](#)())
- string [randomDate](#) ()
Генерация случайной даты
- string [randomTime](#) ()
Генерация случайного времени
- string [randomFlightNumber](#) ()
Генерация случайного номера рейса
- string [randomAirline](#) ()
Генерация случайной авиакомпании
- int [randomPassengers](#) ()
Генерация случайного числа пассажиров
- [FlightData generateFlight](#) ()
Генерация одного случайного рейса
- void [saveToCSV](#) (const vector< [FlightData](#) > &flights, const string &filename)
Сохранение массива рейсов в CSV-файл
- int [main](#) ()
Главная функция генератора данных

Variables

- `const vector< string > AIRLINES`
Список авиакомпаний для генерации
- `const vector< string > PREFIXES = {"SU", "S7", "FV", "DP", "U6", "UT", "N4", "WZ"}`
Префиксы для номеров рейсов
- `random_device rd`
Генератор случайных чисел (Mersenne Twister).

5.1.1 Detailed Description

Генератор тестовых данных для лабораторной работы

Author

[Ваше имя] @student_id [Номер студенческого] @group [Номер группы]

Date

2026

Version

1.0

Создаёт CSV-файлы с информацией об авиарейсах для последующего тестирования алгоритмов сортировки.

5.1.2 Использование

Запустите программу без аргументов:

```
./generate_data.exe
```

5.1.3 Выходные данные

Файлы сохраняются в папку `data/` с именами `flights_{size}.csv`

See also

[main.cpp](#)

5.1.4 Function Documentation

5.1.4.1 gen()

```
mt19937 gen (
    rd() )
```

5.1.4.2 generateFlight()

[FlightData](#) generateFlight ()

Генерация одного случайного рейса

Returns

[FlightData](#) со случайными полями

5.1.4.3 main()

int main ()

Главная функция генератора данных

Returns

0 при успешном выполнении

Последовательно генерирует файлы для размеров: 100, 500, 1000, 5000, 10000, 20000, 30000, 50000, 75000, 100000

Postcondition

Все файлы сохраняются в папку data/

See also

[main.cpp](#)

5.1.4.4 randomAirline()

string randomAirline ()

Генерация случайной авиакомпании

Returns

Название из списка AIRLINES

5.1.4.5 randomDate()

string randomDate ()

Генерация случайной даты

Returns

Строка в формате YYYY-MM-DD

Диапазон: 2024-01-01 до 2025-12-28

5.1.4.6 randomFlightNumber()

```
string randomFlightNumber ()
```

Генерация случайного номера рейса

Returns

Строка вида "SU1234"

5.1.4.7 randomPassengers()

```
int randomPassengers ()
```

Генерация случайного числа пассажиров

Returns

Число от 0 до 300

5.1.4.8 randomTime()

```
string randomTime ()
```

Генерация случайного времени

Returns

Строка в формате HH:MM:SS

Диапазон: 00:00:00 до 23:59:59

5.1.4.9 saveToCSV()

```
void saveToCSV (
    const vector< FlightData > & flights,
    const string & filename)
```

Сохранение массива рейсов в CSV-файл

Parameters

flights	Массив данных
filename	Имя выходного файла

Создаёт файл с заголовком и данными. Если файл существует, перезаписывает его.

5.1.5 Variable Documentation

5.1.5.1 AIRLINES

```
const vector<string> AIRLINES
```

Initial value:

```
= {  
    "Aeroflot", "S7 Airlines", "Rossiya", "Pobeda",  
    "Ural Airlines", "Utair", "Nordwind", "Red Wings"  
}
```

Список авиакомпаний для генерации

5.1.5.2 PREFIXES

```
const vector<string> PREFIXES = {"SU", "S7", "FV", "DP", "U6", "UT", "N4", "WZ"}
```

Префиксы для номеров рейсов

5.1.5.3 rd

```
random_device rd
```

Генератор случайных чисел (Mersenne Twister).

5.2 main.cpp File Reference

```
#include <iostream>  
#include <fstream>  
#include <string>  
#include <vector>  
#include <algorithm>  
#include <chrono>  
#include <iomanip>  
#include <sstream>
```

Classes

- class [Flight](#)
Класс, представляющий информацию об авиарейсе

Functions

- `template<typename T>`
`void bubbleSort (vector< T > &arr)`
Сортировка пузырьком (Bubble sort).
- `template<typename T>`
`void shakerSort (vector< T > &arr)`
Шейкер-сортировка (Shaker sort) - улучшенный вариант пузырька
- `template<typename T>`
`int partition (vector< T > &arr, int low, int high)`
Разбиение массива для быстрой сортировки
- `template<typename T>`
`void quickSortRecursive (vector< T > &arr, int low, int high)`
Рекурсивная реализация быстрой сортировки
- `template<typename T>`
`void quickSort (vector< T > &arr)`
Быстрая сортировка (Quick sort).
- `template<typename T>`
`bool isSorted (const vector< T > &arr)`
Проверка, отсортирован ли массив
- `vector< Flight > loadFromCSV (const string &filename)`
Загрузка данных из CSV-файла
- `void saveResultToFile (const vector< Flight > &arr, const string &filename, const string &sortName)`
Сохранение отсортированного массива в файл
- `template<typename SortFunc>`
`long long measureTime (vector< Flight > arr, SortFunc sortFunc, const string &sortName, const string &resultFilename)`
Замер времени выполнения сортировки
- `int main ()`
Главная функция программы

5.2.1 Function Documentation

5.2.1.1 bubbleSort()

```
template<typename T>
void bubbleSort (
    vector< T > & arr)
```

Сортировка пузырьком (Bubble sort).

Template Parameters

T	Тип элементов массива (должен поддерживать operator>)
---	---

Parameters

arr	Сортируемый массив (передается по ссылке)
-----	---

Алгоритм последовательно проходит по массиву, сравнивая соседние элементы и меняя их местами, если они стоят в неправильном порядке.

@complexity Время: $O(n^2)$ в худшем и среднем случае @complexity Память: $O(1)$ дополнительной

@stable Да (не меняет порядок равных элементов) @natural Да (работает быстрее на частично отсортированных данных)

Warning

На массивах размером > 10000 работает очень медленно

See also

[shakerSort](#)

[quickSort](#)

5.2.1.2 isSorted()

```
template<typename T>
bool isSorted (
    const vector< T > & arr)
```

Проверка, отсортирован ли массив

Template Parameters

T	Тип элементов
---	---------------

Parameters

arr	Проверяемый массив
-----	--------------------

Returns

true если массив отсортирован по возрастанию

@complexity $O(n)$

5.2.1.3 loadFromCSV()

```
vector< Flight > loadFromCSV (
    const string & filename)
```

Загрузка данных из CSV-файла

Parameters

filename	Имя файла (путь)
----------	------------------

Returns

vector<Flight> Загруженные объекты

Ожидает файл с заголовком: "Номер рейса,Авиакомпания,Дата прилета,Время прилета,Число пассажиров"

@error Если файл не открывается, возвращает пустой вектор

See also

[saveResultToFile](#)

5.2.1.4 main()

```
int main ()
```

Главная функция программы

Returns

0 при успешном выполнении

Выполняет следующие шаги:

1. Создаёт папку для результатов
2. Для каждого размера массива (100..100000): а. Загружает данные из CSV б. Запускает каждый алгоритм сортировки с. Замеряет время d. Сохраняет результаты
3. Сохраняет сводную таблицу времени в timing_results.csv

Precondition

Должны существовать файлы data/flights_{size}.csv (генерируются generate_data.exe)

Postcondition

Создаются файлы:

- timing_results.csv (сводная таблица)
- sorted_results/sorted_results_{size}.txt (детальные результаты)

See also

[generate_data.cpp](#)

5.2.1.5 measureTime()

```
template<typename SortFunc>
long long measureTime (
    vector< Flight > arr,
    SortFunc sortFunc,
    const string & sortName,
    const string & resultFilename)
```

Замер времени выполнения сортировки

Template Parameters

SortFunc	Тип функции сортировки
----------	------------------------

Parameters

arr	Массив для сортировки (копия)
sortFunc	Функция сортировки
sortName	Название алгоритма (для вывода)
resultFilename	Имя файла для сохранения результата

Returns

long long Время выполнения в микросекундах

Создаёт копию массива, замеряет время выполнения сортировки, проверяет корректность и сохраняет результат в файл.

See also

[isSorted](#)
[saveResultToFile](#)

5.2.1.6 partition()

```
template<typename T>
int partition (
    vector< T > & arr,
    int low,
    int high)
```

Разбиение массива для быстрой сортировки

Template Parameters

T	Тип элементов
---	---------------

Parameters

arr	Массив
low	Левая граница
high	Правая граница

Returns

Индекс опорного элемента после разбиения

Выбирает опорный элемент (медиану), переставляет элементы так, что все меньшие опорного оказываются слева, а большие — справа.

5.2.1.7 quickSort()

```
template<typename T>
void quickSort (
    vector< T > & arr)
```

Быстрая сортировка (Quick sort).

Template Parameters

T	Тип элементов массива
---	-----------------------

Parameters

arr	Сортируемый массив
-----	--------------------

Реализует алгоритм "разделяй и властвуй":

1. Выбирает опорный элемент
2. Разбивает массив на две части
3. Рекурсивно сортирует каждую часть

@complexity Время: $O(n \log n)$ в среднем, $O(n^2)$ в худшем @complexity Память: $O(\log n)$ под стек рекурсии

@stable Нет @natural Частично (зависит от выбора опорного элемента)

Warning

На почти отсортированных данных может деградировать до $O(n^2)$

See also

std::sort

5.2.1.8 quickSortRecursive()

```
template<typename T>
void quickSortRecursive (
    vector< T > & arr,
    int low,
    int high)
```

Рекурсивная реализация быстрой сортировки

Template Parameters

T	Тип элементов
---	---------------

Parameters

arr	Массив
low	Левая граница
high	Правая граница

See also

[quickSort](#)

5.2.1.9 saveResultToFile()

```
void saveResultToFile (
    const vector< Flight > & arr,
    const string & filename,
    const string & sortName)
```

Сохранение отсортированного массива в файл

Parameters

arr	Отсортированный массив
filename	Имя выходного файла
sortName	Название алгоритма сортировки (для заголовка)

Файл открывается в режиме добавления (ios::app), поэтому результаты разных алгоритмов для одного размера собираются в одном файле.

5.2.1.10 shakerSort()

```
template<typename T>
void shakerSort (
    vector< T > & arr)
```

Шейкер-сортировка (Shaker sort) - улучшенный вариант пузырька

Template Parameters

T	Тип элементов массива
---	-----------------------

Parameters

arr	Сортируемый массив
-----	--------------------

Отличается от пузырька тем, что проходы выполняются в обе стороны: сначала лёгкие элементы всплывают вверх, затем тяжёлые опускаются вниз.

@complexity Время: $O(n^2)$ в худшем случае @complexity Память: $O(1)$ дополнительной

@stable Да @natural Да (лучше пузырька на почти отсортированных данных)

See also

[bubbleSort](#)